PRACTICAL SOLUTION OF NODE JS

AN ISO 9001:2015 CERTIFIED INSTITUTE

COSMOS

The Real Training Centre

ANSWERSHEET OF NODE JS

1) Introduction

- Basic node commands

- v, --version (It is used to print node's version.)
- -h, --help (It is used to print node command line options.)
- -c, --check (Syntax check the script without executing.)

2) Create first node js file.

- index.js

```
var a=10; var b=20;
console.log(a+b);
```

- how to run file

```
node index.js
```

3) Implement basic of JS. (Loop, Array, Conditions and Function)

- index.js

```
let arr = [4,65,34,67,23,7];
let res = arr.filter((item)=>{
        return item;
})
console.log(res);
for (let i = 0; i < 10; i++)
{
        console.log(i);
}
if(a===10)
{
        console.log("Yes it is 10");
}
else
{
        console.log("No it not 10");
}
```

4) Demonstrate to Import File.

- app.js

```
module.exports={
x:10,
y=20,
z:function(){
        return 10;
}
}
```

- index.js

```
const app = require('./app');
console.log(res);
```

5) How to get input from command prompt?

- index.js

```
const fs = require('fs');
const input = process.argv;
if(input[2]=='add')
{
        fs.writeFileSync(input[3],input[4]);
}
else if(input[2]=='remove')
{
        fs.unlinkSync(input[3])
}
else
{
        console.log("invalid input");  }
```

6) How to display list of files in directory?

- index.js

```
const fs = require('fs');
const path = require('path');
const dirPath = path.join(__dirname,'files');
 for (let i = 0; i < 5; i++){
 fs.writeFileSync(dirPath+"/hello"+i+".txt","s ample file");
 }
fs.readdir(dirPath,(err,files)=>{ files.forEach((item)=>{
        console.log("file name is ",item);
        })
})
```

7) Demonstrate CRUD operation with file system.

- index.js

```
const fs = require('fs');
const path = require('path');
const dirPath = path.join(__dirname,'crud');
const filePath = `${dirPath}/apple.txt`;
// CREATE FILE
// fs.writeFileSync(filePath,"This is sample file");
// READ FILE
// fs.readFile(filePath,'utf8',(err,item)=>{
//        console.log(item);
// })
// UPDATE FILE
// fs.appendFile(filePath,'updated content',(err)=>{
//        if(!err) console.log("file is updated");
// })
// RENAME FILE
```

```
// fs.rename(filePath,`${dirPath}/fruit.txt`,(err)=>{
//         if(!err)console.log("file name updated");
// })
// DELETE FILE
// fs.unlinkSync(`${dirPath}/fruit.txt`);
```

8) Learn how to store data using local storage in Node.js.

```
const { LocalStorage } = require('node-localstorage');
// Create a local storage instance
const localStorage = new LocalStorage('./scratch');
// Storing data in local storage
localStorage.setItem('name', 'John Doe');
localStorage.setItem('age', '30');
// Retrieving and printing stored data
const name = localStorage.getItem('name');
const age = localStorage.getItem('age');
console.log(`Name: ${name}`);
console.log(`Age: ${age}`);
```

9) Update Data in Local Storage.

```
const { LocalStorage } = require('node-localstorage');
// Create a local storage instance
const localStorage = new LocalStorage('./scratch');
// Storing initial data
localStorage.setItem('name', 'John Doe');
localStorage.setItem('age', '30');
// Updating age
localStorage.setItem('age', '31');
// Retrieving and printing updated data
const name = localStorage.getItem('name');
const updatedAge = localStorage.getItem('age');
console.log(`Name: ${name}`);
console.log(`Updated Age: ${updatedAge}`);
```

10) Deleting Data from Local Storage.

```
const { LocalStorage } = require('node-localstorage');
// Create a local storage instance
const localStorage = new LocalStorage('./scratch');
// Storing initial data
localStorage.setItem('name', 'John Doe');
localStorage.setItem('age', '30');
// Deleting age from local storage
localStorage.removeItem('age');
// Retrieving and printing remaining data
const name = localStorage.getItem('name');
const age = localStorage.getItem('age');
console.log(`Name: ${name}`);
console.log(`Age: ${age ? age : 'Age not available'}`);
```

11) Clear All Data from Local Storage.

```
const { LocalStorage } = require('node-localstorage');
// Create a local storage instance
const localStorage = new LocalStorage('./scratch');
// Storing multiple pieces of data
localStorage.setItem('name', 'John Doe');
localStorage.setItem('age', '30');
localStorage.setItem('email', 'john@example.com');
// Clearing all data
localStorage.clear();
// Checking if data exists
const name = localStorage.getItem('name');
const age = localStorage.getItem('age');
const email = localStorage.getItem('email');
console.log(`Name: ${name}`);
console.log(`Age: ${age}`);
console.log(`Email: ${email}`);
```

12) Check If Data Exists in Local Storage.

```
const { LocalStorage } = require('node-localstorage');
// Create a local storage instance
const localStorage = new LocalStorage('./scratch');
  // Check if the age exists in local storage
let age = localStorage.getItem('age');
if (!age) {
 // If age doesn't exist, store it
 localStorage.setItem('age', '30');
 age = localStorage.getItem('age');
}
console.log(`Age: ${age}`);
```

13) Demonstrate creating a basic server.

- index.js

```
const http = require('http');
function makeServer(req,resp)
{
        resp.write("<h1>Hello server</h1>"); resp.end();
}
http.createServer(makeServer).listen(4500);
```

14) Install nodemon Package.

- command for installing nodemon package

```
npm i nodemon
```

15) Demonstrate creating basic API.

- index.js

```
const http = require('http'); http.createServer((req,resp)=>
{
        resp.writeHead(200,{'Content- Type':'application\json'});
        resp.write(JSON.stringify({name:'test',email:'test.com'}));
         resp.end();
}).listen(5000);
```

16) How to handle asynchronous data?

- index.js

```
let a = 10;
let b = 0;
setTimeout(() => { b=20;
}, 2000);
console.log(a+b);
```

17) Install express js package.

- npm i express

18) Create basic routing with parameters.

- index.js

```
const express = require('express');
const app = express();
app.get('',(req,res)=>{
        res.send(req.query.name);
});
app.get('/about',(req,res)=>{
        res.send("This is about page");
});
app.get('/contact',(req,res)=>{
        res.send("This is contact page");
});
app.listen(5000);
```

- Pass parameter in URL like localhost:5000/?name=ram

19) Render HTML and JSON using express js.

- index.js

```
const express = require('express');
const app = express();
app.get("",(req,res)=>{
        res.send("<a href='/about'>about</a>");
});
app.get("/about",(req,res)=>{
        res.send("<a href='/'>home</a>");
});
app.listen(5000);
```

20) Render HTML files in express js.

- add basic HTML pages like index.html, about.html, contact.html

- index.js

```
const express = require('express');
const path = require('path');
const app = express();
const publicPath = path.join(__dirname,'public');
app.use(express.static(publicPath));
app.listen(5000);
```

21) Remove extension (.html) from URL.

- index.js

```
const express = require('express');
const app = express();
app.get("",(req,res)=>{
        res.send("<a href='/about'>about</a>");
});
app.get("/about",(req,res)=>{
        res.send("<a href='/'>home</a>");
});
app.listen(5000);
```

22) Demonstarate basic middleware.

- index.js

```
const express = require('express');
const app = express();
const reqFilter = (req,res,next)=>{
if(!req.query.age)
{
        res.send("Please provide age");         }
else if(req.query.age<18)
{
        res.send("You cannot vote");
}else
{
        next();             }
}
app.use(reqFilter);
app.get('/',(req,res)=>{
        res.send("Welcome to my site");
});
app.get('/about',(req,res)=>{ res.send("This is about us page");
}); app.listen(5000);
```

23) Demonstarate router level middleware.

- Middleware.js

```
module.exports = (req, resp, next) => {
if (!req.query.age)
{
        resp.send("Please provide your age")            }
else if (req.query.age<18)
{
        resp.send("You are under aged")
}else
{
        next();
}       }
```

- index.js

```
const express = require('express');
const reqFilter = require('./middleware');
const app = express();
const route = express.Router();
route.use(reqFilter);
app.get('/',(req,res)=>{ res.send("Welcome to Home page");
});
app.get('/users',(req,res)=>{ res.send("Welcome to User page");
});
route.get('/about',(req,res)=>{ res.send("Welcome to About page");
});
route.get('/contact',(req,res)=>{ res.send("Welcome to Contact page");
});
app.use('/',route); app.listen(5000);
```

24) Install mongo compass in your system.

- visit https://www.mongodb.com/try/download/compass

25) Implement basic commands of mongodb.

>> show dbs (to display all database)
>>use <databasename> (to switch specific database)
>>db.createCollection('data') (to create collection)
>>db.data.drop() (to remove collection
'data')
>>show collection (to display all collection from db)
>>db.dropDatabase() (to remove specific database)

26) Install mongodb package.

- npm i mongodb

27) How to connect mongodb.

- index.js

```
const {MongoClient} = require('mongodb') const url='mongodb://127.0.0.1:27017';
const databaseName='e-comm';
const client= new MongoClient(url);
async function getData()
{
        let result = await client.connect();
        db= result.db(databaseName);
        collection = db.collection('products');
        let data = await collection.find({}).toArray();
        console.log(data);
}
getData();
```

28) Read data from mongodb.

- index.js

```
//option-1
const dbConnect= require('./mongodb');

dbConnect().then((resp)=>{
        resp.find({name:'m40'}).toArray().then ((data)=>{
        console.log(data)
})
})
//option-2
const main=async ()=>{
        let data = await dbConnect();
        data = await data.find({name:'m40'}).toArray();
        console.log(data)
}
main();
```

- mongodb.js

```
const {MongoClient} = require('mongodb');
const url='mongodb://127.0.0.1:27017';
const databaseName='e-comm';
const client= new MongoClient(url);
async function dbConnect()
{
        let result = await client.connect();
        db= result.db(databaseName);
        return db.collection('products');
}
module.exports= dbConnect;
```

29) Insert data in mongodb.

- insert.js

```
const dbConnect = require('./mongodb');
const insertData=async ()=>{
        let data = await dbConnect();
        let result = await data.insertMany( [
                {name:'max 5',brand:'micromax',price:420,category:'mo bile'},
                {name:'max 6',brand:'micromax',price:520,category:'mo bile'},
                {name:'max 7',brand:'micromax',price:620,category:'mo bile'},
        ]
        )
        if(result.acknowledged)
        {
                console.warn("data is inserted")
        }
}
insertData();
```

- index.js

```
//option-1
const dbConnect= require('./mongodb');
dbConnect().then((resp)=>{
        resp.find({name:'m40'}).toArray().then ((data)=>{
        console.log(data)
})
})
//option-2
const main=async ()=>{
        let data = await dbConnect();
        data = await data.find({name:'m40'}).toArray();
        console.log(data)
}
main();
```

- mongodb.js

```
const {MongoClient} = require('mongodb');
const url='mongodb://127.0.0.1:27017';
const databaseName='e-comm';
const client= new MongoClient(url);
async function dbConnect()
{
        let result = await client.connect();
        db= result.db(databaseName);
        return db.collection('products');
}
module.exports= dbConnect;
```

30) update data in mongodb.

- update.js

```
const dbConnect= require('./mongodb')
const updateData=async ()=>{
        let data = await dbConnect();
        let result = await data.updateMany(
                { name:'max 5'},
                {
                        $set:{name:'max pro 5', price:1000}
                }
        )
        console.log(result)
        }
updateData();
```

- insert.js

```
const dbConnect = require('./mongodb');
const insertData=async ()=>{
        let data = await dbConnect();
        let result = await data.insertMany( [
                {name:'max 5',brand:'micromax',price:420,category:'mo bile'},
                {name:'max 6',brand:'micromax',price:520,category:'mo bile'},
                {name:'max 7',brand:'micromax',price:620,category:'mo bile'},
```

```
        ]
        )
        if(result.acknowledged)
        {
                console.warn("data is inserted")
        }
}
insertData();
```

- index.js

```
//option-1
const dbConnect= require('./mongodb');
dbConnect().then((resp)=>{
        resp.find({name:'m40'}).toArray().then ((data)=>{
        console.log(data)
})
})
//option-2
const main=async ()=>{
        let data = await dbConnect();
        data = await data.find({name:'m40'}).toArray();
        console.log(data)
}
main();
```

- mongodb.js

```
const {MongoClient} = require('mongodb');
const url='mongodb://127.0.0.1:27017';
const databaseName='e-comm';
const client= new MongoClient(url);
async function dbConnect()
{
        let result = await client.connect();
        db= result.db(databaseName);
        return db.collection('products');
}
module.exports= dbConnect;
```

31) Delete data in mongodb.

- delete.js

```
const dbConnect = require('./mongodb');
const deleteData=async ()=>{
        let data = await dbConnect();
        let result = await data.deleteMany({name:'max 7'});
        console.log(result)
}
deleteData();
```

- update.js

```
const dbConnect= require('./mongodb')
const updateData=async ()=>{
        let data = await dbConnect();
        let result = await data.updateMany(
```

```
                { name:'max 5'},
                {
                        $set:{name:'max pro 5', price:1000}
                }
        )
        console.log(result)
        }
updateData();
```

- insert.js

```
const dbConnect = require('./mongodb');
const insertData=async ()=>{
        let data = await dbConnect();
        let result = await data.insertMany( [
                {name:'max 5',brand:'micromax',price:420,category:'mo bile'},
                {name:'max 6',brand:'micromax',price:520,category:'mo bile'},
                {name:'max 7',brand:'micromax',price:620,category:'mo bile'},
        ]
        )
        if(result.acknowledged)
        {
                console.warn("data is inserted")
        }
}
insertData();
```

- index.js

```
//option-1
const dbConnect= require('./mongodb');
dbConnect().then((resp)=>{
        resp.find({name:'m40'}).toArray().then ((data)=>{
        console.log(data)
})      })
//option-2
const main=async ()=>{
        let data = await dbConnect();
        data = await data.find({name:'m40'}).toArray();
        console.log(data)
}
main();
```

- mongodb.js

```
const {MongoClient} = require('mongodb');
const url='mongodb://127.0.0.1:27017';
const databaseName='e-comm';
const client= new MongoClient(url);

async function dbConnect()
{
        let result = await client.connect();
        db= result.db(databaseName);
        return db.collection('products');
}module.exports= dbConnect;
```

32) Create GET API with mongodb.

- mongodb.js

```
const {MongoClient} = require('mongodb');
const url='mongodb://127.0.0.1:27017';
const databaseName='e-comm';
const client= new MongoClient(url);
async function dbConnect()
{
        let result = await client.connect();
        db= result.db(databaseName);
        return db.collection('products');
}
module.exports= dbConnect;
```

- api.js

```
const dbConnect = require('./mongodb');
const express = require('express');
const app = express();
app.use(express.json());
app.get('/',async (res,resp)=>{
        let data = await dbConnect();
        data= await data.find().toArray();
        resp.send(data);
});
app.post("/", async (req,resp)=>{ let data = await dbConnect();
let result = await data.insertOne(req.body) resp.send(result)
})
app.listen(5000);
```

33) Create POST API with mongodb.

- mongodb.js

```
const {MongoClient} = require('mongodb');
const url='mongodb://127.0.0.1:27017';
const databaseName='e-comm';
const client= new MongoClient(url);
async function dbConnect()
{
        let result = await client.connect();
        db= result.db(databaseName);
        return db.collection('products');
}
module.exports= dbConnect;
```

- api.js

```
const dbConnect = require('./mongodb');
const express = require('express');
const app = express();
app.use(express.json());
app.get('/',async (res,resp)=>{
        let data = await dbConnect();
        data= await data.find().toArray();
```

```
        resp.send(data);
});
app.post("/", async (req,resp)=>{ let data = await dbConnect();
let result = await data.insertOne(req.body) resp.send(result)
})
app.listen(5000)
```

34) Create PUT API with mongodb.

- mongodb.js

```
const {MongoClient} = require('mongodb');
const url='mongodb://127.0.0.1:27017';
const databaseName='e-comm';
const client= new MongoClient(url);
async function dbConnect()
{
        let result = await client.connect();
        db= result.db(databaseName);
        return db.collection('products');
}
module.exports= dbConnect;
```

- api.js

```
const dbConnect = require('./mongodb');
const express = require('express');
const app = express();
app.use(express.json());
app.get('/',async (res,resp)=>{
        let data = await dbConnect();
        data= await data.find().toArray();
        resp.send(data);
});
app.post("/", async (req,resp)=>{ let data = await dbConnect();
let result = await data.insertOne(req.body) resp.send(result)
});
app.put("/:name",async (req,res)=>{ let data = await dbConnect();
        let result = data.updateOne(
                {name:req.params.name},
                {
                        $set:{price:999}
                }
        );
        res.send({result:"update"});
});
app.listen(5000)
```

35) Create DELETE API with mongodb.

- mongodb.js

```
const {MongoClient} = require('mongodb');
const url='mongodb://127.0.0.1:27017';
const databaseName='e-comm';
```

```
const client= new MongoClient(url);
async function dbConnect()
{
        let result = await client.connect();
        db= result.db(databaseName);
        return db.collection('products');
}
module.exports= dbConnect;
```

- api.js

```
const dbConnect = require('./mongodb');
const express = require('express');
const mongodb = require('mongodb');
const app = express();
app.use(express.json());
app.get('/',async (res,resp)=>{
        let data = await dbConnect();
        data= await data.find().toArray();
        resp.send(data);
});
app.post("/", async (req,resp)=>{
        let data = await dbConnect();
        let result = await data.insertOne(req.body);
        resp.send(result)
});
app.put("/:name",async (req,res)=>{
        let data = await dbConnect();
        let result = data.updateOne(
                {name:req.params.name},
                {
                        $set:{price:999}
                }
        });
        res.send({result:"update"});
});
app.delete("/:id",async (req,res)=>{
        console.log(req.params.id);
        const data = await dbConnect();
        const result = data.deleteOne({
                _id:new mongodb.ObjectId(req.params.id)
        });
        res.send(result);
});
app.listen(5000)
```

36) Install mongoose package.

- npm i mongoose

37) Demonstrate CRUD operation with mongoose.

- index.js

```
const mongoose = require('mongoose');
mongoose.connect('mongodb://127.0. 0.1:27017/e-comm');
const productSchema = new mongoose.Schema({
        name: String,
        price: Number,
        brand: String,
        category: String
});
const saveInDB = async () => {
        const Product = mongoose.model('products', productSchema);
        let data = new Product({
                name: "max 100",
                price: 200,
                brand: 'maxx',
                category: 'Mobile'
        });
        const result = await data.save();
        console.log(result);
}
const updateInDB =async () => {
        const Product = mongoose.model('products', productSchema);
        let data =await Product.updateOne(
                { name: "max 6" },
                {
                        $set: {
                                price: 99990,
                                name:'max pro 6'
                        }
        });
        console.log(data);
}
const deleteInDB = async ()=>{
        const Product = mongoose.model('products', productSchema);

        let data = await Product.deleteOne({name:'m40'});
        console.log(data);
}
const findInDB = async ()=>{
        const Product = mongoose.model('products', productSchema);
        let data = await Product.find({name:'max pro 5'});
        console.log(data);
}
saveInDB();
```

38) Create GET API with mongoose.

- config.js

```
const mongoose = require('mongoose');
mongoose.connect("mongodb://127.0. 0.1:27017/e-comm");
```

- product.js

```
const mongoose = require('mongoose');
const productSchema= mongoose.Schema({
        name:String,
        price:Number,
        brand:String,
        category:String
});
module.exports= mongoose.model("products",productSchema);
```

- index.js

```
const express = require('express');
require("./config");
const Product = require('./product');
const app = express();
app.use(express.json());
app.get("/list", async (req, resp) => {
        let data = await Product.find();
        resp.send(data);
});
app.listen(5000);
```

39) Create POST API with mongoose.

- config.js

```
const mongoose = require('mongoose');
mongoose.connect("mongodb://127.0. 0.1:27017/e-comm");
```

- product.js

```
const mongoose = require('mongoose');
const productSchema= mongoose.Schema({
        name:String,
        price:Number,
        brand:String,
        category:String
});
module.exports= mongoose.model("products",productSchema);
```

- index.js

```
const express = require('express');
require("./config");
const Product = require('./product');
const app = express();
app.use(express.json());
app.post("/create", async (req, resp) => {
```

```
        let data = new Product(req.body);
        const result = await data.save();
        resp.send(result);
});
app.listen(5000);
```

40) Create PUT API with mongoose.

- config.js

```
const mongoose = require('mongoose');
mongoose.connect("mongodb://127.0. 0.1:27017/e-comm");
```

- product.js

```
const mongoose = require('mongoose');
const productSchema= mongoose.Schema({
        name:String,
        price:Number,
        brand:String,
        category:String
});
module.exports= mongoose.model("products",productSchema);
```

- index.js

```
const express = require('express');
require("./config");
const Product = require('./product');
const app = express();
app.use(express.json());
app.put("/update/:_id",async (req, resp) => {
        console.log(req.params)
        let data = await Product.updateOne(
                req.params,
                {
                        $set: req.body                  }
        );
        resp.send(data);
});
app.listen(5000);
```

41) Create DELETE API with mongoose.

- config.js

```
const mongoose = require('mongoose');
mongoose.connect("mongodb://127.0. 0.1:27017/e-comm");
```

- product.js

```
const mongoose = require('mongoose');
const productSchema= mongoose.Schema({
        name:String,
        price:Number,
        brand:String,
        category:String
});
module.exports= mongoose.model("products",productSchema);
```

- index.js

```
const express = require('express');
require("./config");
const Product = require('./product');
const app = express();
app.use(express.json());
app.delete("/delete/:_id", async (req, resp) => {
        console.log(req.params);
        let data = await Product.deleteOne(req.params);
        resp.send(data);
});
app.listen(5000);
```

42) Create search API with mongodb.

- index.js

```
const express = require('express');
require("./config");
const Product = require('./product');
const app = express();
app.use(express.json());
app.get("/search/:key",async (req,resp)=>{
        let data = await Product.find(
        {
                "$or":[
                        {
                                name:{
                                        $regex:req.params.key
                                }
                        },
                        {
                                brand:{
                                        $regex:req.params.key
                                }
                        }
                ]
        });
        resp.send(data);
});
app.listen(5000);
```

43) Install MySQL package.

- npm i mysql

44) Create GET API with MySQL.

- config.js

```
const mysql = require("mysql");
const con= mysql.createConnection({
        host:'localhost',
        user:'root',
```

```
        password:"",
        database:"test"
});
con.connect((err)=>{
        if(err)
        {
                console.warn("error in connection")
        }
});
module.exports = con;
```

- index.js

```
const express = require("express");
const con = require("./config");
const app = express();
app.get("/", (req, resp) => {
        con.query("select * from students", (err, result) => {
                if(err)
                {
                        resp.send("error in api");
                }
                else
                {
                        resp.send(result);
                }
        });
});
app.listen("5000");
```

45) Create POST API with MySQL.

- config.js

```
const mysql = require("mysql");
const con= mysql.createConnection({
        host:'localhost',
        user:'root',
        password:"",
        database:"test"
});
con.connect((err)=>{
        if(err)
        {
                console.warn("error in connection")
        }
});
module.exports = con;
```

- index.js

```
const express = require("express");
const con = require("./config");
const app = express();
app.use(express.json());
app.post("/",(req,resp)=>{
```

```
        const data=req.body;
        con.query("INSERT INTO students SET?",data,(error,results,fields)=>{
                if(error)
                {
                        throw error;
                }
                resp.send(results)
        });
});
app.listen("5000");
```

46) Create PUT API with MySQL.

- config.js

```
const mysql = require("mysql");
const con= mysql.createConnection({
        host:'localhost',
        user:'root',
        password:"",
        database:"test"
});
con.connect((err)=>{
        if(err)
        {
                console.warn("error in connection")
        }
});
module.exports = con;
```

- index.js

```
const express = require("express");
const con = require("./config");
const app = express();
app.use(express.json());
        app.put("/:id",(req,resp)=>{
        const data= [req.body.name,req.body.age,req.para ms.id];
        console.log(data);
        con.query("UPDATE students SET student_name = ?, age = ? WHERE id = ?",
data,(error,results,fields)=>{
                if(error)
                {
                        throw error;
                }
                resp.send(results)
        });
});
app.listen("5000");
```

47) Create DELETE API with MySQL.

- config.js

```
const mysql = require("mysql");
```

```
const con= mysql.createConnection({
        host:'localhost',
        user:'root',
        password:"",
        database:"test"
});
con.connect((err)=>{
        if(err)
        {
                console.warn("error in connection")
        }
});
module.exports = con;
```

- index.js

```
const express = require("express");
const con = require("./config");
const app = express();
app.use(express.json());
app.delete("/:id",(req,resp)=>{
        const data= [req.params.id];
        con.query("DELETE FROM students WHERE id = ?", data,(error,results,fields)=>{
                if(error)
                {
                        throw error;
                }
                resp.send(results)
        });
});app.listen("5000");
```

48) Install multer package.

- npm i multer

49) Upload files using multer.

- index.js

```
const express = require('express');
const multer = require('multer');
const app = express();
const upload = multer({
        storage:multer.diskStorage({
                destination:function(req,file,cb){
                        cb(null,"uploads");
                },
                filename:function(req,file,cb){
                        cb(null,file.fieldname+"-"+Date.now()+".jpg");        }});
}).single("user_file");
app.post("/upload",upload,(req,res)=>{
        res.send("file upload");
});
app.listen(5000);
```